Lab 1 - Greedy heuristics

Author: Michał Żurawski 160252

Description of the problem

Each instance contains nodes with coordinates (x, y) and a cost. Select exactly half of the nodes (rounded up) and connect them into a closed cycle. Minimise **cycle length + sum of the costs of the selected nodes**. Distances are Euclidean, rounded to the nearest integer.

Algorithms

Three greedy construction methods, 200 runs each (run i starts from node i), and 200 random solutions as a baseline. All greedy methods add one node per step, the one that increases the objective the least, and differ only in **where the new node may be inserted**.

Intuition

A greedy method builds the solution one node at a time and never undoes a choice. "Nearest" here means the smallest increase of the objective, so both the added length and the node cost count: a close but expensive node can lose to a farther cheap one.

The more places a node may go, the better each step can be. Nearest neighbor (end only) can only extend the path at its end, so it walks away from the start and pays a long closing edge at the end. Inserting at any position lets a node slip in between two tour nodes it lies close to. Greedy cycle keeps the tour closed all the time, so the closing edge is already paid for in every step.

Example: the path ends at c, and node u lies 40 away from c but right next to the edge (a, b). End only must pay 40, any position pays d(a, u) + d(u, b) − d(a, b), e.g. 5.

Core calculations

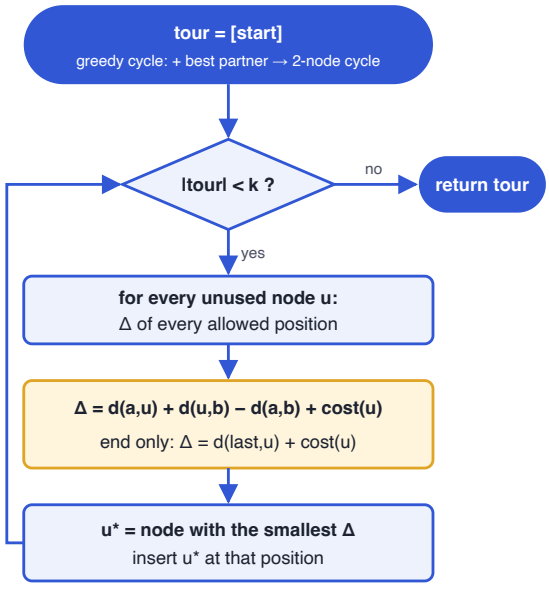
For every unused node u, the objective increase of adding it:

Between tour nodes a and b:
`Δ = d(a, u) + d(u, b) − d(a, b) + cost(u)`

In front of the path: `Δ = d(u, first) + cost(u)`

After the path (the only option of end only): `Δ = d(last, u) + cost(u)`

The (node, position) with the smallest Δ is inserted. The path is closed into a cycle at the end.



Method	Start	u may go
Random	k random nodes	–
NN (end only)	[start]	after the last node
NN (any position)	[start]	front, between nodes, end
Greedy cycle	[start, best partner]	into any cycle edge

Best partner = the node minimising `d(start, v) + cost(v)`.

Pseudocode

Simplified from `python/solver.py`. The three greedy methods share the loop; they differ in the starting tour and in how the next node and its position are found.

```
def greedy(start):
    tour = [start]                                      # greedy cycle: + nearest partner
    while len(tour) < k:
        node, position = cheapest_insertion(tour)       # end only: nearest_unused_node(tour[-1]), at the end
        tour.insert(position, node)
    return tour

def cheapest_insertion(tour):
    for u in unused_nodes:
        length_increase, position = cheapest_position(tour, u)   # in the path or in the cycle
        value = length_increase + cost[u]
    return the u with the smallest value, and its position

def random_solution(seed):
    shuffle(nodes)
    return nodes[:k]
```

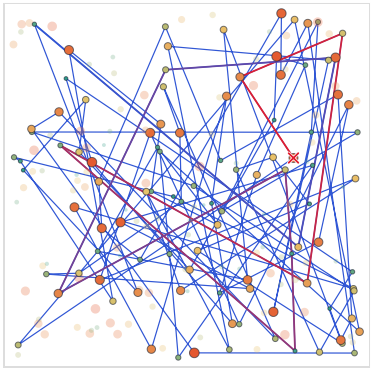
Results

Objective = cycle length + sum of node costs, over 200 runs per method. The best value in each column is in bold.
All best solutions were checked with the solution checker (the same computation as the lecturer's Solution checker.xlsx): **yes**.
Results are identical in all implemented languages: **yes**.

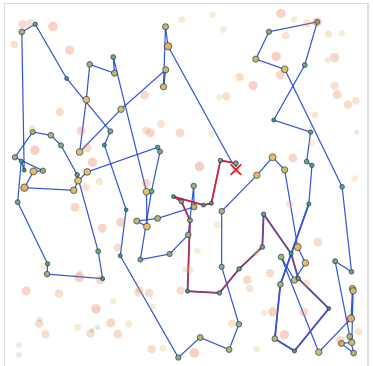
Instance TSPA

Method	Min	Max	Avg
Random solution	237131	287973	263514
Nearest neighbor (end only)	84362	90075	87477
Nearest neighbor (any position)	**72328**	77022	74347
Greedy cycle	72590	**75281**	**73797**

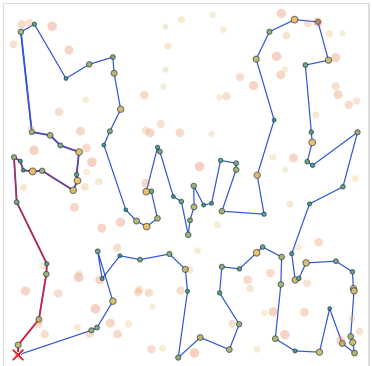
Visualisation (Instance TSPA)



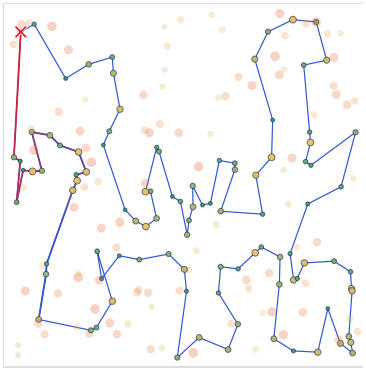
Random solution
237131 (length 140636 + cost 96495)



Nearest neighbor (end only)
84362 (length 31319 + cost 53043)



Nearest neighbor (any position)
72328 (length 21816 + cost 50512)

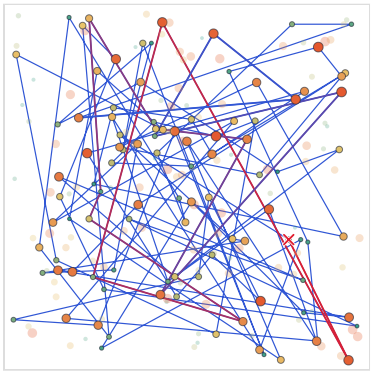


Greedy cycle
72590 (length 21570 + cost 51020)

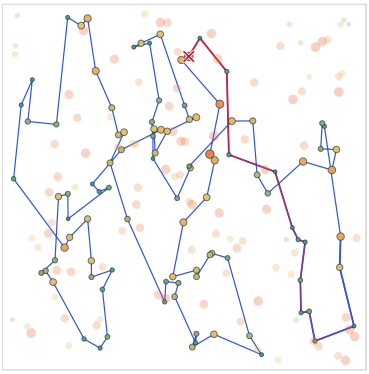
Instance TSPB

Method	Min	Max	Avg
Random solution	188214	231879	206802
Nearest neighbor (end only)	56007	**63166**	**59397**
Nearest neighbor (any position)	**52915**	66164	59475
Greedy cycle	53016	63736	59709

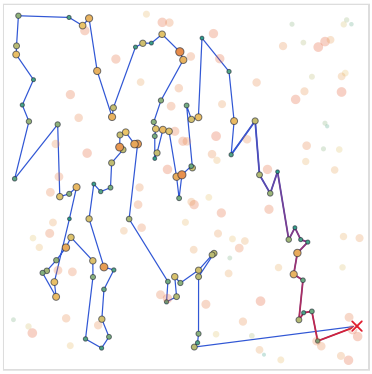
Visualisation (Instance TSPB)



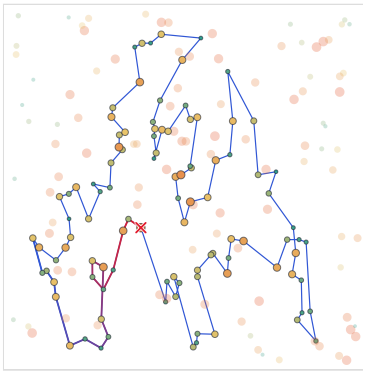
Random solution
188214 (length 138112 + cost 50102)



Nearest neighbor (end only)
56007 (length 24632 + cost 31375)



Nearest neighbor (any position)
52915 (length 20068 + cost 32847)



Greedy cycle
53016 (length 16704 + cost 36312)

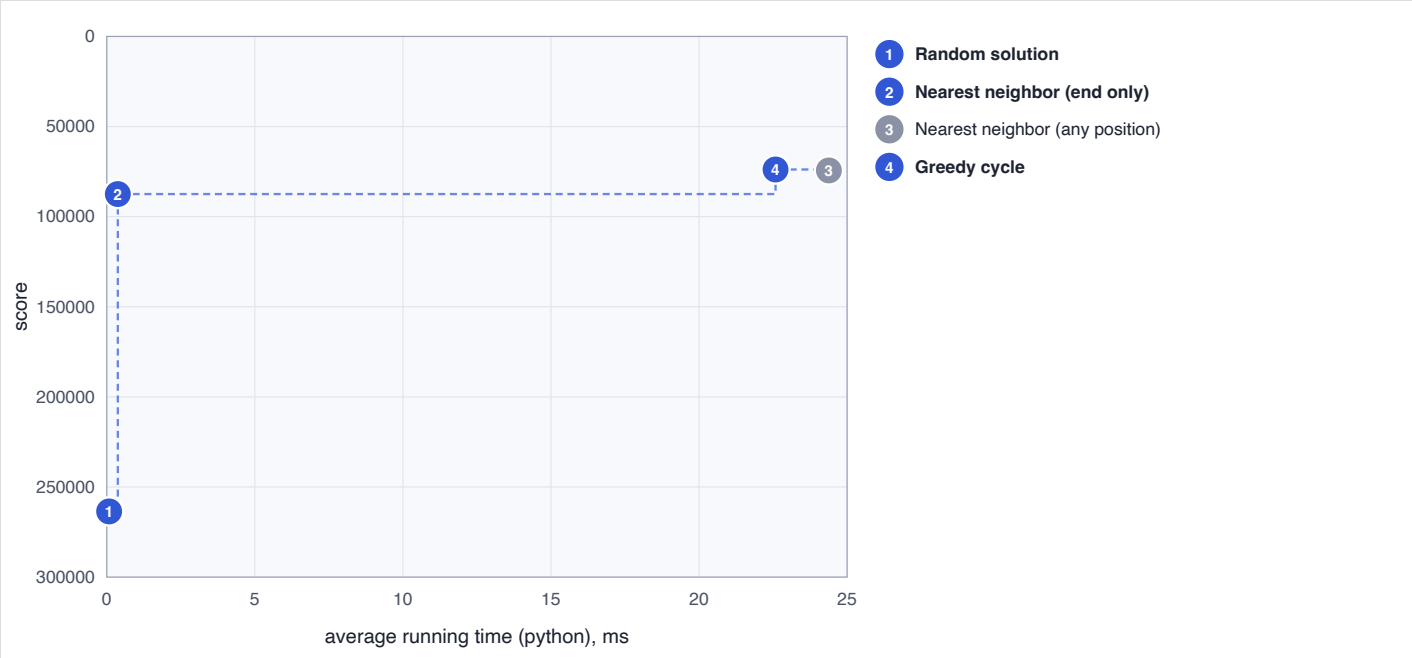
Running time

Average time of one run of the algorithm only, over the same runs as above. The fastest language in each row is in bold.

Instance TSPA

Method	Python	C	C++	Rust
Random solution	77.6 µs	380 ns	321 ns	**295 ns**
Nearest neighbor (end only)	374.0 µs	**14.0 µs**	20.4 µs	23.5 µs
Nearest neighbor (any position)	24.39 ms	534.2 µs	**481.1 µs**	567.2 µs
Greedy cycle	22.58 ms	687.4 µs	**462.6 µs**	648.7 µs

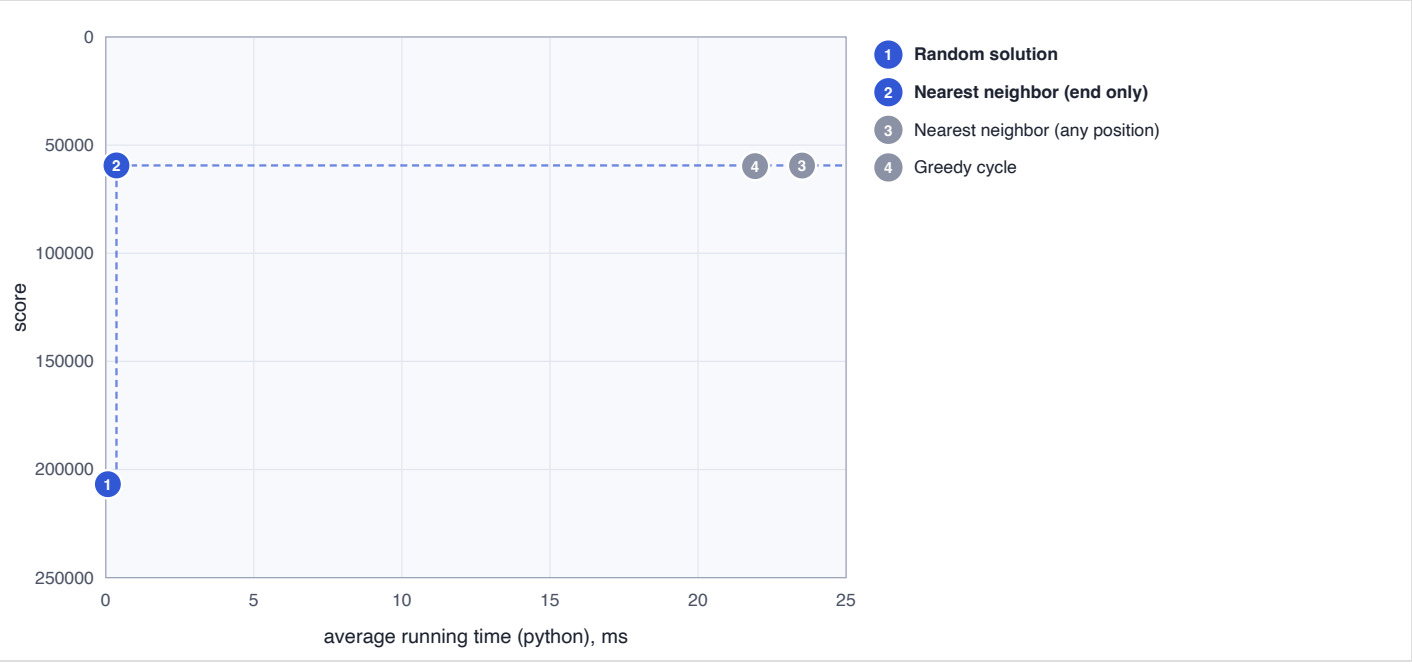
Pareto plot (Instance TSPA)



Instance TSPB

Method	Python	C	C++	Rust
Random solution	70.2 µs	**425 ns**	481 ns	500 ns
Nearest neighbor (end only)	360.0 µs	**15.6 µs**	24.0 µs	18.3 µs
Nearest neighbor (any position)	23.51 ms	529.4 µs	**515.9 µs**	561.6 µs
Greedy cycle	21.92 ms	541.8 µs	**464.2 µs**	639.2 µs

Pareto plot (Instance TSPB)



Best solutions (0-based node indices)

Instance TSPA

Random solution
128, 34, 55, 103, 152, 57, 106, 85, 120, 112, 86, 43, 48, 96, 63, 38, 100, 61, 164, 195, 178, 194, 58, 117, 6, 158, 173, 40, 37, 121, 156, 192, 5, 101, 165, 1, 29, 131, 147, 60, 52, 170, 64, 54, 142, 93, 41, 88, 107, 159, 80, 68, 20, 36, 15, 22, 12, 146, 67, 116, 149, 189, 137, 133, 35, 99, 51, 30, 13, 154, 122, 75, 155, 33, 115, 162, 23, 87, 197, 56, 19, 78, 118, 82, 66, 182, 139, 190, 144, 161, 127, 65, 191, 50, 114, 45, 16, 26, 163, 129

Nearest neighbor (end only)
54, 9, 33, 36, 177, 67, 144, 53, 135, 58, 134, 176, 174, 106, 89, 107, 57, 182, 151, 93, 50, 3, 1, 164, 178, 29, 136, 186, 184, 47, 159, 130, 153, 78, 88, 193, 170, 146, 75, 64, 150, 42, 113, 43, 85, 142, 8, 131, 84, 158, 192, 181, 114, 141, 180, 132, 76, 80, 48, 31, 63, 149, 168, 97, 127, 187, 30, 123, 152, 108, 60, 116, 49, 11, 119, 160, 83, 90, 51, 16, 35, 87, 45, 140, 52, 179, 185, 28, 102, 156, 157, 71, 91, 38, 100, 4, 112, 139, 115, 5

Nearest neighbor (any position)
104, 129, 69, 127, 97, 168, 116, 149, 63, 11, 49, 160, 83, 123, 100, 152, 108, 60, 31, 48, 80, 91, 157, 71, 4, 76, 132, 180, 28, 102, 185, 35, 156, 51, 16, 67, 144, 140, 53, 179, 52, 177, 36, 33, 9, 54, 131, 174, 8, 164, 186, 136, 163, 29, 101, 178, 1, 169, 3, 50, 147, 47, 93, 106, 42, 89, 113, 130, 159, 146, 170, 153, 193, 78, 88, 107, 64, 57, 182, 151, 150, 176, 72, 134, 84, 58, 158, 192, 181, 114, 135, 73, 45, 87, 141, 187, 30, 189, 2, 171

Greedy cycle
31, 116, 149, 168, 63, 11, 49, 60, 108, 152, 100, 90, 123, 83, 160, 97, 127, 69, 171, 2, 189, 30, 187, 141, 87, 45, 73, 135, 114, 181, 192, 158, 58, 84, 134, 72, 176, 150, 151, 182, 57, 64, 107, 88, 78, 193, 153, 170, 146, 159, 130, 113, 89, 42, 106, 93, 47, 147, 50, 3, 169, 1, 178, 101, 29, 163, 136, 186, 164, 142, 8, 174, 131, 54, 9, 33, 36, 177, 52, 179, 53, 140, 144, 67, 16, 51, 156, 35, 185, 102, 28, 180, 132, 76, 4, 71, 157, 91, 80, 48

Instance TSPB

Random solution
80, 23, 16, 71, 163, 97, 140, 42, 136, 144, 107, 193, 172, 44, 12, 15, 105, 190, 160, 98, 151, 116, 103, 45, 147, 195, 177, 60, 110, 64, 153, 55, 73, 106, 188, 137, 17, 170, 50, 8, 13, 6, 88, 104, 0, 133, 38, 184, 61, 4, 113, 5, 68, 20, 111, 134, 129, 39, 191, 59, 148, 54, 63, 158, 96, 21, 198, 171, 95, 142, 1, 181, 27, 186, 34, 51, 161, 18, 75, 46, 130, 154, 56, 77, 176, 162, 139, 199, 57, 90, 2, 91, 185, 19, 168, 40, 132, 84, 66, 30

Nearest neighbor (end only)
32, 89, 57, 76, 54, 127, 129, 90, 153, 20, 14, 2, 110, 118, 56, 114, 196, 10, 148, 143, 78, 192, 162, 104, 185, 184, 3, 151, 69, 172, 133, 120, 96, 121, 198, 48, 72, 46, 86, 117, 160, 136, 8, 21, 154, 140, 47, 170, 125, 177, 6, 132, 29, 197, 175, 134, 45, 60, 40, 71, 155, 97, 115, 180, 65, 28, 149, 145, 158, 116, 147, 42, 50, 130, 30, 183, 38, 64, 85, 59, 167, 51, 87, 179, 82, 88, 171, 99, 67, 24, 34, 181, 152, 73, 98, 27, 135, 75, 79, 156

Nearest neighbor (any position)
110, 2, 14, 20, 74, 153, 182, 52, 90, 129, 127, 80, 54, 192, 162, 104, 76, 185, 57, 89, 86, 46, 72, 3, 184, 173, 151, 122, 117, 8, 160, 69, 172, 136, 133, 120, 156, 43, 48, 96, 198, 121, 130, 191, 50, 42, 147, 116, 37, 146, 13, 149, 28, 145, 65, 158, 177, 125, 26, 170, 197, 36, 132, 29, 6, 180, 115, 155, 71, 175, 134, 45, 157, 60, 40, 141, 97, 154, 140, 47, 38, 21, 176, 30, 183, 189, 168, 64, 59, 85, 51, 73, 167, 152, 34, 24, 181, 179, 87, 82

Greedy cycle
49, 64, 33, 40, 60, 71, 155, 141, 157, 45, 134, 175, 62, 36, 197, 29, 132, 178, 103, 6, 180, 115, 170, 177, 125, 26, 97, 140, 154, 47, 38, 21, 176, 30, 183, 191, 130, 63, 121, 198, 96, 48, 89, 156, 120, 133, 136, 172, 160, 69, 8, 117, 72, 46, 86, 184, 3, 173, 122, 151, 98, 111, 27, 135, 76, 185, 57, 104, 162, 54, 192, 127, 52, 182, 153, 20, 74, 2, 14, 90, 129, 80, 150, 84, 66, 67, 31, 24, 34, 152, 181, 88, 179, 87, 82, 73, 167, 51, 59, 85

Source code

labs/lab1 (Python, C, C++, Rust)

Conclusions

- Random solutions are about 3.5 times worse than any greedy method.
- Inserting at any position beats appending at the end on TSPA; on TSPB all greedy methods are close on average.
- Insertion at any position runs about 30 times longer than end only, but still well under 1 ms per run.